Deep Neural Network Surrogates for Approximate Bayesian Computation: A Simulation Study for Mutation-Rate Estimation in Markov Branching Processes

Sachin Letchumanan[1*], Hongxiao Zhu[1] and Xiaowei Wu[1]

[1*]Department of Statistics, Virginia Tech, Blacksburg, VA, 24061, USA.

*Corresponding author(s). E-mail(s): sachin.letchumanan@gmail.com;
Contributing authors: hongxiao@vt.edu; xwwu@vt.edu;

Abstract

Approximate Bayesian computation (ABC) enables likelihood-free inference for stochastic simulators, but its per-iteration cost makes posterior analysis infeasible when the simulator is slow, as for the branching-process models used to estimate mutation rates from Luria–Delbrück-style fluctuation experiments. A standard remedy substitutes a Gaussian-process (GP) surrogate for the simulator inside an ABC Markov chain Monte Carlo (MCMC) sampler (GPS-ABC), but its cubic fitting cost limits how much simulator output the surrogate can use, and in the four-parameter model of the original study a single chain takes days. We ask whether a neural-network surrogate, calibrated by split-conformal prediction, can make the same substitution, and what it buys. A one-dimensional benchmark shows tighter calibrated intervals and a 68- to 2,000-fold speedup over exact ABC-MCMC. The main study is the four-parameter two-stage model with differential growth, run in the original study's own regime after checking that our data generator is statistically indistinguishable from theirs. There the summary statistic is exactly zero for 30% of data sets and its two features are strongly correlated; a zero-aware network fits it at the data's noise floor where a GP fit to the original design's **1,500** points is **1.45** times above it (**2.4** at **300** points), and needs only that design size. On the same data sets, against the original authors' own sampler, it matches the accuracy of GPS-ABC for the identified parameters with posterior intervals for the second mutation probability **1.7** to **3.6** times shorter, in about forty seconds per data set where a GPS-ABC chain takes days. An amortized neural posterior estimator trained on the same simulations agrees on what the data identify, the second mutation probability and loosely the growth parameter and not the first mutation probability or the transition time, and is better calibrated over prior draws, while the surrogate-based samplers are sharper and mildly overconfident. Code, data and a reproducible pipeline accompany the paper.

Keywords: approximate Bayesian computation; likelihood-free inference; surrogate modeling; conformal prediction; heteroscedastic regression; Markov branching process

1 Introduction

Approximate Bayesian computation (ABC) makes Bayesian inference possible for stochastic models whose likelihood is intractable but which can

be simulated forward (Beaumont et al, 2002; Marjoram et al, 2003; Marin et al, 2012). The basic ABC-MCMC sampler (Marjoram et al, 2003) accepts a proposed parameter θ' in proportion to how closely a simulated summary statistic matches the observed one. Every iteration costs at least one forward simulation, so when the simulator is expensive the per-iteration cost dominates the analysis and can put a full posterior exploration out of reach.

Lu et al (2023) study this problem for a two-type Markov branching process (MBP) model of microbial mutation, used to estimate mutation probabilities from Luria–Delbrück-style fluctuation experiments. Their exact simulator (Algorithm 2) tracks the process cell by cell, and a single call takes from a fraction of a second to several minutes. To make ABC-MCMC tractable they apply *GPS-ABC* (Meeds and Welling, 2014): fit a Gaussian process (GP) regression surrogate to a modest design of simulator outputs, then use the GP's predictive mean and variance in place of the simulator inside the acceptance step. This is the source of their reported two-to-three-order-of-magnitude speedup.

The GP brings a scaling limit that is well known and, here, binding. Fitting an n-point GP costs $O(n^3)$ and a query costs $O(n)$ in the dense implementation used at this problem size. Lu et al (2023) accordingly work with about 50 design points in the one-dimensional case and 1,500 in their four-parameter extension, and note that GPS-ABC can be *less* accurate than plain ABC-MCMC when the design must be built from the expensive exact simulator. We ask:

> *Can a neural-network surrogate replace the Gaussian process inside the same ABC-MCMC procedure, and under what conditions does it help?*

A feedforward network has no cubic fitting term and its query cost is flat in the size of its training set. It also gives up something the GP provides for free: a predictive variance. Inside the ABC acceptance kernel that variance is consumed directly, so a network must be built to produce one and then calibrated (Section 3). That requirement, rather than the regression itself, makes the substitution non-trivial.

Contributions. The paper's centre of gravity is the four-parameter two-stage model of Lu et al's Simulation Study 2 (Study II); the one-dimensional model (Study I) is the benchmark that shows where a surrogate matters.

(i) *A calibrated, zero-aware neural surrogate.* A heteroscedastic network with split-conformal calibration (Lei et al, 2018) that substitutes for the GP in Lu et al's sampler with no other change to the inference algorithm. In the two-stage model the summary statistic is exactly zero on about 30% of data sets and the two features $(\bar{d}, s)$ it is built from are strongly correlated, so the network is a hurdle model for a pair of correlated features rather than a regression on $\log_{10} \bar{d}$.

(ii) *A like-for-like comparison with the original sampler.* We run the original authors' MATLAB GPS-ABC on the same simulated data sets, in the original study's regime ($t_p = 20$, $p \in [10^{-11}, 10^{-7}]$, four parameters), after checking that our data generator is statistically indistinguishable from theirs at their own design points. A control trained on only the original design size separates the effect of more simulation from the effect of the surrogate.

(iii) *Identifiability, not the surrogate, limits accuracy.* The data identify p_2 and, loosely, δ, and not p_1 or τ. A neural posterior estimator (NPE) trained on the same simulations and the surrogate-based sampler agree on this, which makes it a property of the summary statistic and not of either method.

(iv) *Calibration diagnostics that do not saturate.* Coverage at a fixed truth is 1.000 for any nearly unidentified parameter because the interval spans the prior. We use simulation-based calibration over truths drawn from the prior (Talts et al, 2018) together with posterior contraction: NPE is calibrated and less sharp, the surrogate-based samplers sharper and mildly overconfident.

(v) *A one-dimensional benchmark.* The one setting where surrogate error does bind: the network's input-dependent variance yields tighter, still-calibrated intervals than the GP, and DNN-ABC is 68 to over 2,000 times faster than exact ABC-MCMC.

(vi) *Negative results we report rather than tune away.* More training data does not help once the surrogate is at the noise floor (1,500

design points are as good as 10,000), residual networks are no better than plain MLPs, and modelling the correlation of the two features improves the likelihood but does not restore nominal coverage.

Related work. GPS-ABC is one instance of a line of work using Gaussian processes to reduce simulator calls in likelihood-free inference: Wilkinson (2014) models the ABC log-likelihood surface directly, Gutmann and Corander (2016)'s BOLFI uses Bayesian optimization to place the next expensive simulation, and Järvenpää et al (2021); Järvenpää and Corander (2023) extend this to noisy, parallel and predictive settings. All inherit the $O(n^3)$ fitting cost; none substitutes a neural surrogate for the GP, which is the substitution studied here. A separate literature avoids the GP-versus-network choice by learning the posterior (or likelihood, or likelihood ratio) directly with a neural density estimator trained on simulator draws (Papamakarios and Murray, 2016; Greenberg et al, 2019; Cranmer et al, 2020), implemented in toolkits such as `sbi` (Tejero-Cantero et al, 2020). These methods are *amortized*: training is paid once and new data sets need no further simulation or MCMC, which GPS-ABC, DNN-ABC and exact ABC-MCMC cannot offer. We include NPE in Study II, the higher-dimensional study, to make that trade-off concrete and to test what the sampler and the surrogate's variance model add on top of the same simulations; Section 5.5 gives the reasons in full. The coverage question is motivated by Talts et al (2018) and Hermans et al (2022), who report that posterior approximations routinely pass regression-level diagnostics yet fail to cover the truth at the stated rate when checked directly, and by Fan and White (2026), who show that a narrow NPE proposal can give a biased posterior that still reproduces the observed summaries. Section 4.4's distinction between the surrogate's regression coverage and the ABC posterior's coverage is an instance of the same failure mode.

The paper proceeds as follows. Section 2 reviews the model and GPS-ABC, Section 3 develops the surrogate, and Sections 4 and 5 report the two studies. Section 6 discusses implications and limitations and Section 7 describes the reproducible pipeline.

2 Background: ABC for the Markov Branching Process Model

2.1 The mutation model and the observed statistic

Following Lu et al (2023), a population initiated by a single wild-type cell grows for a fixed observation period under a two-type Markov branching process (MBP). Three quantities govern it: the per-division mutation probability p, the division-rate parameter a (which sets the time scale), and the mutant relative growth rate δ. We take $a = 1$ throughout and, in Study I, $\delta = 1$ as well; Study II frees δ. We refer to Lu et al (2023) for the full construction and for their Algorithm 2 (an exact, cell-by-cell simulator) and Algorithm 4 (a fast approximate simulator). We re-implement both in Python and check the result against the training data the original authors released (Section 3.3).

An experiment consists of J independently plated parallel cultures, each grown from one wild-type cell for the same fixed time. Culture j yields a final wild-type count Z_j and mutant count X_j. Following Lu et al, we work with the scalar summary

$$\bar{d} \;=\; \frac{1}{J}\sum_{j=1}^{J}\sqrt{X_j/Z_j},$$

and, for the two-stage model, its fourth-root analogue and the standard deviation of the same values (Section 5.1). Every regression below models $\log_{10}\bar{d}$ rather than $\bar{d}$, because $\log_{10}\bar{d}$ varies smoothly and close to linearly in $\theta \equiv \log_{10} p$ (Lu et al, 2023, Figure 2), a target both GP and neural-network regression fit considerably better.

2.2 ABC-MCMC and GPS-ABC

Write $\theta = \log_{10} p$. Given an observed statistic $\bar{d}_{\text{obs}}$, classical ABC-MCMC (Marjoram et al, 2003) targets

$$\pi_\epsilon(\theta \mid \bar{d}_{\text{obs}}) \;\propto\; \pi(\theta)\,\mathbb{E}_{S\sim F_\theta}\left[K_\epsilon\big(\bar{d}_{\text{obs}}, S\big)\right],$$

with a smoothing kernel K_ϵ of bandwidth ϵ. The "exact" baseline throughout this paper is not this literal kernel average but Wood's (Wood, 2010) *synthetic likelihood*: at each proposed θ' we draw n_s pilot simulations, take their sample mean

$m(\theta')$ and variance $v(\theta')$, and score $\bar{d}_{\text{obs}}$ against a matched Gaussian,

$$\hat{\pi}_\epsilon(\bar{d}_{\text{obs}} \mid \theta') \;=\; \mathcal{N}\big(\bar{d}_{\text{obs}};\; m(\theta'),\; \epsilon^2 + v(\theta')\big)\,.$$

This is exactly the likelihood form the surrogates use, with a predicted variance $\sigma_\phi(\theta')^2$ in place of $v(\theta')$, so every column of the result tables shares one likelihood and differs only in how $(m(\theta), v(\theta))$ is obtained. Algorithm 1 states the shared Metropolis–Hastings sampler: a truncated-exponential prior on θ (uniform in the multi-parameter study) and a truncated-normal random-walk proposal.

Algorithm 1 ABC-MCMC (Metropolis–Hastings, synthetic likelihood)

1: Initialize θ_0; set tolerance ϵ, simulator-repeat count n_s.
2: **for** $t = 1, \ldots, T$ **do**
3: Propose $\theta' \sim q(\cdot \mid \theta_{t-1})$ (truncated-normal random walk)
4: Obtain $(m(\theta'), v(\theta'))$: either (a) simulate n_s times from $F_{\theta'}$ and take their sample mean and variance, or (b) evaluate a surrogate's predicted mean and variance $(\mu_\phi(\theta'), \sigma_\phi(\theta')^2)$ (Section 3)
5: Compute acceptance probability

$$\alpha \;=\; \min\!\left(1,\; \frac{\pi(\theta')\, q(\theta_{t-1} \mid \theta')}{\pi(\theta_{t-1})\, q(\theta' \mid \theta_{t-1})} \cdot \frac{\hat{\pi}_\epsilon(\bar{d}_{\text{obs}} \mid \theta')}{\hat{\pi}_\epsilon(\bar{d}_{\text{obs}} \mid \theta_{t-1})}\right)$$

6: Set $\theta_t = \theta'$ with probability α, else $\theta_t = \theta_{t-1}$
7: **end for**
8: **return** $\{\theta_t\}_{t=B+1}^{T}$ after discarding B burn-in draws

Step 4(a) is the expensive one. *GPS-ABC* (Meeds and Welling, 2014), applied by Lu et al (2023) to this model, instead fits a GP to a design $\{(\theta_i, \bar{d}_i)\}_{i=1}^{n}$ of simulator outputs *once*, and at every iteration uses step 4(b): the GP's predictive mean and variance replace $(m(\theta'), v(\theta'))$ directly. This sets the standard any replacement must meet. It is not enough to predict the mean accurately; the variance must also be calibrated, because the two failure modes are asymmetric and both corrupt the posterior. An overconfident σ inflates the acceptance rate where the surrogate does not know the response, and an underconfident one over-rejects.

The GP's fitting cost is $O(n^3)$ and its per-query cost $O(n)$, since the predictive mean and variance require solving a system with the full training kernel matrix. Lu et al (2023) report about 50 design points in the one-dimensional case and 1,500 initial points in a four-parameter extension, and note that GPS-ABC trained on the exact simulator can be less accurate than uncorrected ABC-MCMC, plausibly because the cubic cost forces a small design when each design point comes from the expensive exact simulator.

3 A Neural-Network Surrogate for ABC

We replace the GP in step 4(b) of Algorithm 1 with a neural network f_ϕ trained to predict both the mean and the variance of $\log_{10} \bar{d}$ as a function of the parameters of interest. The substitution is deliberately minimal: the sampler, the acceptance ratio and the convolution with ϵ^2 are exactly those of GPS-ABC, so any difference in the results is attributable to the surrogate. We call the procedure *DNN-ABC*. The two ingredients that let the network stand in for the GP are an input-dependent predictive variance and a guarantee that it is calibrated.

3.1 Heteroscedastic regression

f_ϕ has two output heads, $\mu_\phi(x)$ and $\log \sigma_\phi^2(x)$, where x is $\theta = \log_{10} p$ alone (Study I) or $(\log_{10} p_1, \log_{10} p_2, \tau, \log_{10} \delta)$ (Study II), with no derived features. Both heads are trained jointly by minimizing the Gaussian negative log-likelihood

$$\mathcal{L}(\phi) \;=\; \frac{1}{n} \sum_{i=1}^{n} \left[\frac{\big(y_i - \mu_\phi(x_i)\big)^2}{2\, \sigma_\phi^2(x_i)} \;+\; \frac{1}{2} \log \sigma_\phi^2(x_i) \right], \tag{1}$$

with $y_i = \log_{10} \bar{d}_i$. Unlike squared-error training, this targets an *input-dependent* predictive variance, the network's analogue of the GP's posterior variance. That is a requirement, not a refinement: the replicate-to-replicate noise in $\log_{10} \bar{d}$ is far from homoscedastic (Sections 4 and 5.4), and a GP with a single noise term cannot represent that shape.

3.2 Split-conformal calibration

A trained variance head approximates the true conditional variance from a finite sample; nothing

in Eq. (1) guarantees it is calibrated. Since the acceptance step consumes it directly, we apply split-conformal calibration (Lei et al, 2018). On a held-out calibration split of size m we form scores

$$s_i \;=\; \frac{|y_i - \mu_\phi(x_i)|}{\sigma_\phi(x_i)}, \qquad i = 1, \ldots, m,$$

and set the calibration factor $\hat{c}$ to the $\lceil (m+1)(1-\alpha) \rceil / m$ empirical quantile of $\{s_i\}$. The calibrated $(1-\alpha)$ predictive interval is $\mu_\phi(x) \pm \hat{c}\,\sigma_\phi(x)$, and for exchangeable calibration and test points it has marginal coverage of at least $1-\alpha$ in finite samples, with no distributional assumption (Lei et al, 2018). We use $\alpha = 0.05$ and call this the surrogate's *regression calibration*, to distinguish it from the empirical coverage of the downstream ABC credible intervals, a different quantity with no equivalent guarantee (Sections 4.4 and 5.10). The calibrated $\hat{c}\,\sigma_\phi(x)$ is also what enters the acceptance kernel, so the uncertainty that is validated is the uncertainty the sampler uses.

3.3 Simulator validation

Every number below rests on our re-implementations of the simulators of Lu et al (2023), so we check them against the original authors' own data. They release the training sets they used for GPS-ABC: for the four-parameter model, 1,500 design points at which they ran their fast simulator five times, and for the constant-rate model, 1,500 points $\times$ ten runs at each of five mutation rates. At each of their design points we draw 400 values from our simulator and compute the randomised probability-integral transform of each value they stored (the statistic has an atom at zero, so ties are randomised). If the two simulators agree at every design point, the transforms are i.i.d. uniform, which a Kolmogorov–Smirnov test checks, overall and within quarters of each parameter's range. The four-parameter data are not rejected (overall $p = 0.08$ on 7,500 values; the smallest of 16 quarter-range p-values is 0.005 at 400 draws and 0.08 at 1,500, above the Bonferroni threshold of 0.003 in both), nor are the constant-rate data ($p = 0.22$, 0.52, 0.69 at $p = 10^{-8}$, 10^{-6}, 10^{-4}; 15,000 values each). The constant-rate exact simulator is additionally checked against the plating-time equation of Lu et al (2023) (agreement to 3×10^{-5} with the R solver). The design points themselves cannot match theirs draw for draw, since a Latin hypercube is random; what we match is the mechanism.

3.4 Simulation design and reporting

We follow the ADEMP structure of Morris et al (2019). *Aims*: compare MOM, MLE, exact ABC-MCMC, GPS-ABC and DNN-ABC in Study I, and GPS-ABC, DNN-ABC and NPE in Study II (the constant-rate MOM and MLE do not apply there, and exact ABC-MCMC is out of reach at $t_p = 20$), on point accuracy, interval precision, calibration and cost. *Data-generating mechanisms*: the simulators of Lu et al (2023), checked against the training data they released (Section 3.3), at the parameter settings of Sections 4.3 and 5.1. *Estimands*: p (via $\theta = \log_{10} p$) in Study I and (p_1, p_2, τ, δ) jointly in Study II. *Performance measures*: MSE and normalized RMSE of $\hat{p}$, mean 95% credible-interval length, empirical coverage, and wall-clock time (per 100 MCMC iterations in Study I, per data set in Study II); for Study II also the distance of each posterior from the GPS-ABC posterior it is meant to replace, simulation-based calibration and posterior contraction (Section 5.10).

Study I is evaluated over $R = 40$ independent replicated data sets per cell. Study II uses 100 observed data sets per truth case for DNN-ABC and NPE and, because one GPS-ABC chain takes days (Section 5.11), the first five of the same data sets for GPS-ABC, so every comparison with it is paired. Every comparison carries its Monte Carlo standard error: for $e_r = (\hat{p}_r - p)^2$, $\widehat{\mathrm{MCSE}} = \mathrm{sd}(e_r)/\sqrt{R}$. For two methods we report the MSE gap in units of its combined MCSE (the Δ/SE column of Table 2). This is a descriptive diagnostic of whether a point comparison is resolved at the replicate count, not a hypothesis test; we call a difference resolved when $|\Delta/\mathrm{SE}| > 2$.

4 Study I: A Benchmark with One Parameter

Study I reproduces Lu et al's constant-rate benchmark: $a = \delta = Z_0 = 1$ and only p varying. Its role in this paper is to show *where* a surrogate matters; the multi-parameter inference question is Study II. Ground truth is 101 log-spaced values of p with $\log_{10} p \in [-8, -2]$, ten replicates each (1,010 rows), $J = 100$. We split *by replicate* rather than by grid

point, so every p appears in every split and a split tests generalisation to fresh noise, with no leakage: replicates 1–5 train (505 rows), 6–8 calibrate and early-stop (303), 9–10 are held out (202). The GPS-ABC baseline is fit on replicates 1–8, so both surrogates see the same data and neither sees the test replicates.

The ABC comparison evaluates observed data sets at $J \in \{10, 50, 100\}$ cultures. Both surrogates are trained separately for each J: for $J \in \{10, 50\}$ a J-culture $\bar{d}$ is rebuilt from the same simulated data by averaging a random J-subset of the 100 recorded per-culture values (five subsets per row), so no new simulator calls are needed. Section 4.4 shows why this matters.

4.1 Architecture

The network is a shallow funnel MLP: input $\theta = \log_{10} p$, two GELU-activated dense layers of width 32 and 16 (626 parameters), and the two linear heads of Section 3.1. It is trained with Adam under early stopping on calibration-split NLL, weight decay 10^{-5} and no dropout. The width is the outcome of a two-stage search (activation and normalisation at a 128–64 baseline, then capacity); full tables are in Supplementary Material S1.

The one decisive finding is BatchNorm (Ioffe and Szegedy, 2015): an otherwise reasonable 128–64 network carrying it fits *eleven times worse* than the same network without it. BatchNorm's per-minibatch statistics inject batch-dependent noise and bias predictions near the hard edge of the one-sided θ domain, so what closes the gap to the GP is removing a component most practitioners include by default. Among BatchNorm-free networks, ReLU, SiLU (Elfwing et al, 2018), GELU (Hendrycks and Gimpel, 2016) and tanh are within 7% of one another (three seeds), a statistical tie; we choose GELU on structural grounds (smoothness, no dead units). A second round with 7-network deep ensembles did not improve on the single network.

Capacity is also nearly irrelevant. A width and depth sweep from 128–64 down to a linear control (three seeds, then confirmed at paper scale: 40 replicates, the full $p \times J$ grid, 600 MCMC iterations) finds 32–16 the best point estimate on curve fit and on the downstream task at 14× fewer parameters, though that margin is not distinguishable from Monte Carlo noise. Every candidate from 8 to 128–64 parameters beats the GP baseline, and only the linear control shows a clear cost. We therefore deploy 32–16 on parsimony, and report every table and figure below for it; the 128–64 network remains in Supplementary Material S1 as the architecture the BatchNorm search ran against. Convolutional and recurrent architectures are not tested here: with one scalar input a 1-D convolution has no neighbours to convolve over and a recurrent cell no second step, so both collapse to a linear map.

4.2 Regression calibration

Table 1 reports the surrogate's fit and split-conformal coverage by split.

Held-out coverage of 0.9554 (193 of 202 points) is within sampling noise of the nominal 0.95, as the guarantee of Lei et al (2018) predicts. Figure 1 shows the calibrated band and compares the network's input-dependent predictive standard deviation with the GP's flat one.

4.3 ABC simulation results

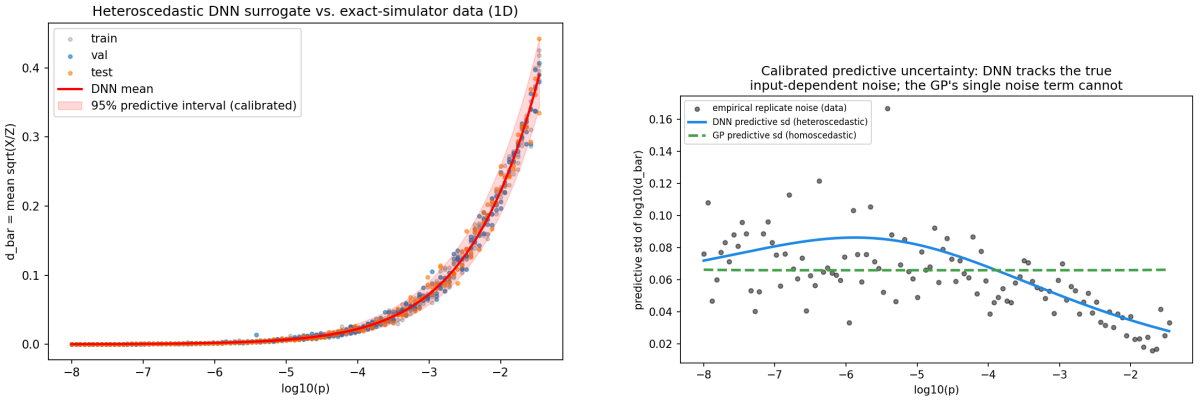
We ran the ABC-MCMC / GPS-ABC / DNN-ABC comparison at 40 replicated data sets per cell, 600 MCMC iterations (250 burn-in), $n_s = 6$ simulator repeats per exact iteration, and prior $\theta \sim$ TruncExp on $[-5, -2]$ (bounded so the exact simulator in the loop stays feasible; the surrogates are trained on $[-8, -2]$ and queried in range), on the grid $p \in \{10^{-4}, 10^{-3}, 10^{-2}\} \times J \in \{10, 50, 100\}$. MOM and MLE follow Lu et al's closed forms as classical baselines.

DNN-ABC's point estimate is lower than GPS-ABC's in five of nine cells and higher in four, but no difference in either direction is resolved: the largest $|\Delta/\mathrm{SE}|$ is 1.44 ($p = 10^{-2}$, $J = 50$, favouring GPS-ABC). This is a statistical tie in every cell, consistent with a GP on a comparable design already being close to optimal for a one-dimensional smooth curve. Both surrogates match, and in several cells slightly improve on, exact ABC-MCMC, and all ABC methods substantially outperform MOM and MLE, reproducing Lu et al's central finding.

DNN-ABC's interval is shorter than GPS-ABC's in all nine cells, 21% tighter on average and up to 46%, with no loss of point accuracy. The margin varies from negligible (0.1% at $p = 10^{-4}$,

Table 1: One-dimensional surrogate: fit and 95% regression coverage by split.

Split	n	MSE($\log_{10} \bar{d}$)	MAE($\log_{10} \bar{d}$)	95% coverage
Train	505	0.00424	0.0515	0.964
Calibration	303	0.00533	0.0538	0.954
Test	**202**	**0.00434**	**0.0525**	**0.9554**



(a) Calibrated 95% predictive band

(b) Input-dependent uncertainty vs. the GP

Fig. 1: One-dimensional surrogate calibration. Left: the calibrated 95% predictive band tracks the response curve across the full range of θ. Right: the network's heteroscedastic predictive standard deviation (which enters the ABC acceptance kernel directly) tracks the true input-dependent replicate noise, while the GP's single homoscedastic noise term is flat by construction.

$J = 100$) to 46% ($p = 10^{-2}$, $J = 100$); against exact ABC-MCMC the network is narrower in seven of nine cells. Whether this is gained precision or overconfidence is not settled by length alone; Section 4.4 measures coverage.

DNN-ABC runs at a flat ≈ 0.13 s per 100 iterations regardless of p or J, because its query cost does not depend on training-set size. GPS-ABC costs ≈ 0.12 s at $J = 100$ (a 550-point design) but ≈ 0.74 s at $J \in \{10, 50\}$ (a 2,750-point design), a $\sim 6\times$ slowdown that tracks the GP's $O(n)$ query cost against the $5\times$ larger design: the clearest direct demonstration in this paper of the scaling difference of Section 1. Exact ABC-MCMC's cost grows with both p and J; DNN-ABC is 68–2381$\times$ faster. Figure 2 shows the three posteriors concentrating on the same true p on a representative data set.

4.4 Posterior coverage

Split-conformal *regression* calibration (Table 1) is a guarantee about the surrogate's predictive interval, not about whether the downstream ABC credible interval brackets the true p 95% of the time. Tables 2 and 3 are silent on the second quantity, so we save every credible interval's endpoints and measure it.

A surrogate trained only on $J = 100$ summaries cannot know that a $J = 10$ observation is noisier. Querying the J=100-only checkpoints at $J = 10$ drops GPS-ABC and DNN-ABC coverage to 0.43–0.53, about half of nominal. The J-specific surrogates of Table 5 recover most of that shortfall: at $J = 10$ they cover at 0.825–0.875, one to three Monte Carlo standard errors below nominal, and both reach or exceed nominal by $J = 100$. Neither surrogate is consistently the more under-covering, so Table 3's length advantage for DNN-ABC is better evidence of real precision than it would be if its intervals were also the least calibrated. Separately, exact ABC-MCMC is itself below nominal at $J = 10$ (0.775–0.850) despite calling the true simulator at the correct J; this is a property of the

Table 2: Study I, Table 1: MSE of $\hat{p}$ across five estimators; normalized RMSE $\sqrt{\text{MSE}}/p$ in parentheses. Bold marks the lower of GPS-ABC and DNN-ABC in each row. Δ/SE is the GPS-ABC−DNN-ABC MSE gap in units of its combined Monte Carlo standard error (Section 3.4); values below ~ 2 in magnitude are not clearly resolved at $R = 40$ replicates.

p	J	MOM	MLE	ABC-MCMC	GPS-ABC	DNN-ABC	Δ/SE
10^{-4}	10	9.36e-9 (0.97)	1.02e-8 (1.01)	5.95e-9 (0.77)	7.01e-9 (0.84)	**5.25e-9 (0.72)**	0.37
10^{-4}	50	8.68e-9 (0.93)	9.59e-9 (0.98)	1.91e-9 (0.44)	1.98e-9 (0.45)	**1.77e-9 (0.42)**	0.45
10^{-4}	100	4.18e-9 (0.65)	4.64e-9 (0.68)	7.75e-10 (0.28)	7.95e-10 (0.28)	**7.68e-10 (0.28)**	0.10
10^{-3}	10	1.38e-6 (1.17)	1.60e-6 (1.26)	4.11e-7 (0.64)	6.46e-7 (0.80)	**4.58e-7 (0.68)**	0.60
10^{-3}	50	3.21e-7 (0.57)	3.85e-7 (0.62)	1.11e-7 (0.33)	**1.11e-7 (0.33)**	1.33e-7 (0.36)	−0.48
10^{-3}	100	2.71e-7 (0.52)	3.29e-7 (0.57)	4.29e-8 (0.21)	4.48e-8 (0.21)	**4.42e-8 (0.21)**	0.04
10^{-2}	10	3.42e-5 (0.58)	4.40e-5 (0.66)	1.67e-5 (0.41)	**1.77e-5 (0.42)**	2.30e-5 (0.48)	−1.02
10^{-2}	50	7.71e-6 (0.28)	1.17e-5 (0.34)	7.01e-6 (0.26)	**5.46e-6 (0.23)**	9.93e-6 (0.32)	−1.44
10^{-2}	100	4.01e-6 (0.20)	6.08e-6 (0.25)	3.05e-6 (0.17)	**3.14e-6 (0.18)**	3.37e-6 (0.18)	−0.21

Table 3: Study I, Table 2: mean length of the 95% ABC credible interval for $\hat{p}$. Bold marks the shorter of GPS-ABC and DNN-ABC.

p	J	ABC-MCMC	GPS-ABC	DNN-ABC
10^{-4}	10	2.62e-4	3.11e-4	**2.45e-4**
10^{-4}	50	1.56e-4	1.50e-4	**1.44e-4**
10^{-4}	100	1.21e-4	1.16e-4	**1.16e-4**
10^{-3}	10	2.80e-3	3.32e-3	**2.45e-3**
10^{-3}	50	1.43e-3	1.66e-3	**1.29e-3**
10^{-3}	100	9.70e-4	1.20e-3	**8.81e-4**
10^{-2}	10	1.83e-2	1.96e-2	**1.89e-2**
10^{-2}	50	1.16e-2	1.74e-2	**1.04e-2**
10^{-2}	100	7.73e-3	1.25e-2	**6.80e-3**

synthetic-likelihood kernel at this culture count, not of any surrogate.

The residual $J = 10$ gap should not be explained away. Split-conformal calibration is a marginal guarantee over the training distribution and does not size the interval correctly at every θ; the $J = 10$ design is the smallest and noisiest of the three. The practical lesson is that a surrogate's interval-length advantage is not by itself evidence of equal calibration, and that training the surrogate at the deployment condition is necessary, though not always sufficient, for a trustworthy ABC posterior.

5 Study II: The Two-Stage Mutation Model

Study I holds $a = \delta = 1$ and a single constant mutation probability. The multi-parameter study of Lu et al (2023), their Simulation Study 2, is a different model, not the same one with more parameters freed. The mutation probability is a step function of time,

$$p(t) = \begin{cases} p_1, & 0 < t \le \tau, \\ p_2, & \tau < t \le t_p, \end{cases} \tag{2}$$

jumping at an unknown transition time τ, and mutants grow at δ times the wild-type rate. The motivation is experimental: mutagenesis under sub-inhibitory antibiotic stress involves a recovery step before plating and so two distinct mutation regimes.

Table 4: Study I, Table 3: wall-clock cost, seconds per 100 MCMC iterations. The final column is the speedup of DNN-ABC over exact ABC-MCMC.

p	J	ABC-MCMC	GPS-ABC	DNN-ABC	Speedup
10^{-4}	10	50.08	0.739	0.130	386×
10^{-4}	50	170.01	0.742	0.127	1342×
10^{-4}	100	301.74	0.126	0.127	2381×
10^{-3}	10	16.33	0.743	0.127	128×
10^{-3}	50	65.32	0.743	0.127	514×
10^{-3}	100	125.75	0.126	0.126	997×
10^{-2}	10	8.52	0.746	0.126	68×
10^{-2}	50	40.07	0.746	0.126	318×
10^{-2}	100	78.39	0.124	0.126	623×

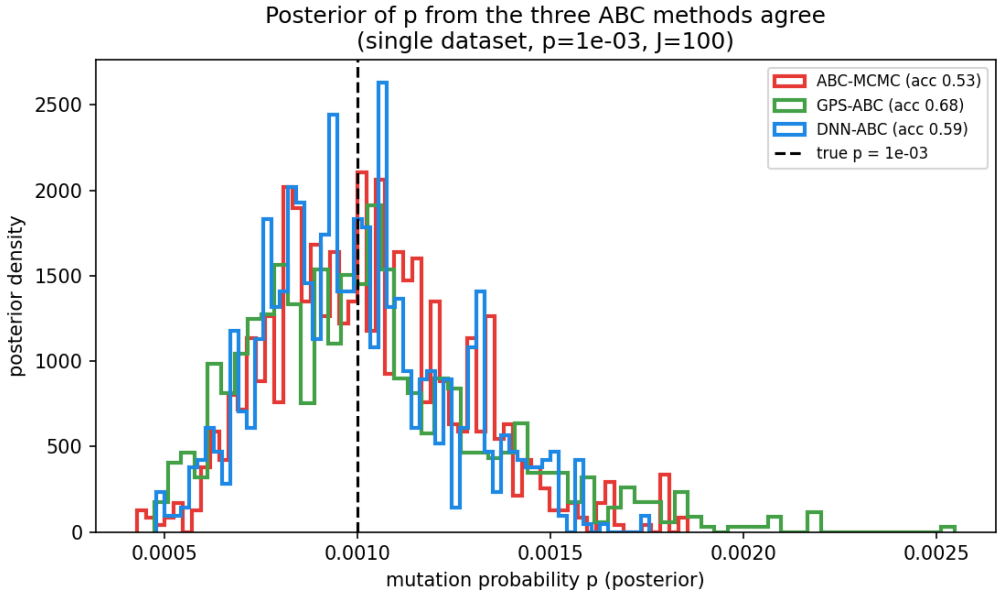


Fig. 2: Study I: ABC-MCMC, GPS-ABC, and DNN-ABC posteriors for θ on a single representative data set.

Study II estimates all four parameters jointly, $\theta = (\log_{10} p_1, \log_{10} p_2, \tau, \log_{10} \delta)$, which is exactly the problem of Lu et al's Simulation Study 2. It is the main study of this paper. It is run in their regime, $t_p = 20$ and $p_1, p_2 \in [10^{-11}, 10^{-7}]$, so the results can be set directly against theirs, and it uses the same construction, summary statistic and sampler as the original GPS-ABC study.

5.1 What is taken from the original study

The comparison with GPS-ABC is only meaningful if the data the two methods see are generated by the same mechanism, so the construction follows Lu et al throughout (Table 6). Where their paper and their released code disagree we follow the paper: the parameter box for τ, the threshold ξ for refining the surrogate, the prior on τ, and the order of the parameters in θ (the code lists $\log_{10} \delta$ first).

Table 5: Study I: empirical posterior coverage of the 95% ABC credible interval for $\hat{p}$, with binomial Monte Carlo standard error ($\sqrt{\hat{c}(1-\hat{c})/R}$, $R = 40$). This is a different quantity from Table 1's regression coverage; see Section 4.4.

p	J	ABC-MCMC	GPS-ABC	DNN-ABC
10^{-4}	10	0.775 (0.066)	0.825 (0.060)	0.850 (0.056)
10^{-4}	50	0.900 (0.047)	0.875 (0.052)	0.900 (0.047)
10^{-4}	100	0.925 (0.042)	0.950 (0.034)	0.950 (0.034)
10^{-3}	10	0.800 (0.063)	0.875 (0.052)	0.875 (0.052)
10^{-3}	50	0.975 (0.025)	0.975 (0.025)	0.925 (0.042)
10^{-3}	100	0.950 (0.034)	0.975 (0.025)	0.925 (0.042)
10^{-2}	10	0.850 (0.056)	0.875 (0.052)	0.875 (0.052)
10^{-2}	50	1.000 (0.000)	1.000 (0.000)	0.925 (0.042)
10^{-2}	100	0.950 (0.034)	1.000 (0.000)	0.925 (0.042)

Table 6: Study II settings. Everything in the first block is the original study's (Section 3.2 and the released code); the second block is what differs because the surrogate is a neural network.

Shared with Lu et al (2023)	
Simulator	fast two-stage simulator (their Algorithm 4 extended as in their code), $Z_0 = 1$, $a = 1$
Plating time, cultures	$t_p = 20$, $J = 100$ cultures per dataset
Statistic	$\sqrt[4]{X_j/Z_j}$ per culture; features $\bar{d}$ (mean) and s (standard deviation) over the J cultures
Parameter box	$\log_{10} p_1, \log_{10} p_2 \in [-11, -7]$, $\tau \in [0.1, 19.9]$, $\log_{10} \delta \in [\log_{10} 0.5, \log_{10} 2]$
Priors	$\mathrm{TN}(\log_{10} \hat{p}_{\mathrm{MOM}}, 20)$ on $\log_{10} p_1, \log_{10} p_2$; $\mathrm{TN}(10, 20)$ on τ; $\mathrm{TN}(0, 20)$ on $\log_{10} \delta$
Sampler	random-walk Metropolis–Hastings, truncated-normal proposals, 20,000 draws, 5,000 burn-in
Truths	$p_1 = 10^{-9}$, $\tau = 14$, $\delta = 0.8$; case 1: $p_2 = 10^{-8}$, case 2: $p_2 = 5 \times 10^{-8}$
Initial design	Latin hypercube over the box
Different, by design	
Design size	10,000 design points × 5 replicates × 100 cultures (GPS-ABC: 1,500 points × 5 single cultures)
Surrogate	zero-aware neural network, trained once; no refinement step

Training data. A Latin hypercube over the box, built by the maximin criterion of MATLAB's `lhsdesign` (best of five random designs; Lu et al cite the optimal-design method of Liao et al (2010)), gives 10,000 design points. Each is simulated five times, a replicate being $J = 100$ independent single-culture draws, so the data set has 50,000 rows and 5×10^6 culture-level draws against the original design's 7,500: 6.7 times as many design points and 667 times as many draws. This is what a surrogate with no cubic fitting cost can use and a Gaussian process cannot, and we control for it in Section 5.7. The whole data set is generated in under a minute on a 32-core server. As a check on the generator we compared it with the training set the original authors ship with their code (1,500 × 5 points in the four-parameter box): evaluated at their design points, our simulator reproduces the fraction of cultures with no mutants (0.564 against 0.558), the mean statistic (0.00751 against 0.00752) and the replicate standard deviation (0.00371 against 0.00365).

Observed data. For each truth case we simulate 100 independent observed data sets of $J = 100$ cultures. Every method, GPS-ABC included, is applied to the same data sets, which is what makes the comparison paired. We cannot reuse the original authors' data sets, which were generated with MATLAB's random-number stream and are not part of their release.

The summary statistic is the fourth root. Lu et al use $\sqrt{X/Z}$ for the constant-rate study but $\sqrt[4]{X/Z}$ for the two-stage model, to make the response curve "smooth and non-flat". Study II

therefore uses $\bar{d} = \frac{1}{J}\sum_j (X_j/Z_j)^{1/4}$ and the standard deviation s of the same J values, the two features their sampler compares. About 30% of simulated data sets have $\bar{d} = 0$ exactly, with no mutant in any of the 100 cultures, and 27% of the design points have $\bar{d} = 0$ in all five replicates; every zero has $p_2 < 10^{-8.8}$, and the proportion of zeros falls from 0.91 for $p_2 \in [10^{-11}, 10^{-10}]$ to 0.30 for $[10^{-10}, 10^{-9}]$ and 0.001 above. Study I's $\log_{10} \bar{d}$ target is therefore undefined on a large part of the box, which shapes the surrogate (Section 5.3).

A caution about the meaning of p. We use the fast simulator exactly as published. At the settings where we could run both, it produces fewer mutants than the cell-by-cell simulator of the same paper (their Algorithms 3 and 4): at $t_p = 9$ with $p = 2 \times 10^{-3}$ a culture has 134 mutants on average under Algorithm 4 and 309 under Algorithm 2, and under the two-stage rates $(10^{-3}, 5\times10^{-3})$ with $\tau = 4$ the means are 198 and 499 (3,000 cultures each). The two use the same symbol p but not the same quantity: Algorithm 4 draws round(pZ) mutation events, while in Algorithm 2 each daughter cell mutates independently with probability p. We did not reconcile them, because every method here sees the same simulator and the same truths, so no comparison among methods depends on it; but p is to be read in the sense of Algorithm 4 when set against Lu et al's Table 4, which uses it too, and not as a per-division probability.

5.2 Identifiability

The data say how much the summary can determine. On the rows with $\bar{d} > 0$ the correlation of $\log_{10} \bar{d}$ with $\log_{10} p_2$ is 0.76, with $\log_{10} p_1$ 0.07, with τ 0.04 and with $\log_{10} \delta$ 0.12: the summary responds strongly to p_2, weakly to δ and p_1 and barely to τ, as in Lu et al's Figures 3–4, which show multimodal p_1 and τ marginals. The reason is structural: mutations arising before τ are rare because few cells exist that early, so p_1's contribution to the mutant count at $t_p = 20$ is swamped by p_2's. A fair evaluation therefore reports error per parameter, on the $\log_{10}$ scale for p_1, p_2: for a weakly identified parameter the posterior mean sits wherever the prior puts its mass, so natural-scale RMSE diverges without conveying anything.

5.3 A zero-aware neural surrogate

The surrogate of Section 3 needs two changes in this setting. The first is the zero mass. Where $\bar{d} = 0$ with positive probability, a Gaussian model of $\log_{10} \bar{d}$ is not defined, and flooring the data at an arbitrary constant makes the surrogate fit the constant. We model the distribution of the features $(\bar{d}, s)$ given θ as a *hurdle* distribution,

$$\begin{aligned} \Pr(\bar{d} = 0 \mid \theta) &= \pi_\phi(\theta), \\ (\log_{10} \bar{d},\ \log_{10} s) \mid \bar{d} > 0, \theta &\sim \mathcal{N}\big(\mu_\phi,\ \operatorname{diag} \sigma_\phi^2\big), \end{aligned} \tag{3}$$

with a logistic head for π_ϕ and two Gaussian mean and log-variance heads. ($s = 0$ exactly when $\bar{d} = 0$.) The likelihood of an observed data set with features $(\bar{d}_{\text{obs}}, s_{\text{obs}})$ is $\pi_\phi(\theta)$ if $\bar{d}_{\text{obs}} = 0$ and otherwise $(1 - \pi_\phi(\theta))\,\mathcal{N}(\log_{10} \bar{d}_{\text{obs}}, \log_{10} s_{\text{obs}}; \mu_\phi, \operatorname{diag} \hat{c}^2 \sigma_\phi^2)$. The second change is the second feature. The GPS-ABC sampler of Lu et al compares both the mean and the standard deviation of the observed statistic, with a separate Gaussian-process surrogate for each. A surrogate for $\bar{d}$ alone would give DNN-ABC less information than the method it replaces, so the network has the two heads for both features. In GPS-ABC they are treated as independent given θ, and our DNN-ABC does the same, so that it differs from GPS-ABC in the surrogate and nothing else. The assumption is not harmless: within a design point, the residuals of $\log_{10} \bar{d}$ and $\log_{10} s$ have correlation 0.61 overall and 0.87 where $p_2 \le 10^{-9}$, because both are driven by the same few large mutant clones. We therefore also fit the surrogate with a correlation head, $\rho_\phi(\theta) = 0.99 \tanh(\cdot)$, replacing the diagonal covariance in Eq. (3) by one with correlation $\rho_\phi(\theta)$ and the same variances, and call that version DNN-ABC-ρ. Its held-out joint negative log-likelihood of the two features is -3.26 against -2.90 for the independent version (nats per data set, same test designs; the fitted mean $|\rho_\phi|$ is 0.60). Training minimises the binary cross-entropy of the logistic head plus the Gaussian negative log-likelihood of Eq. (1) summed over the two features on the rows with $\bar{d} > 0$, with a mean-squared-error warm-up for the first twenty epochs so that the variance head cannot mask a poorly fit mean. The scale factors $\hat{c}$ are split-conformal (Section 3.2), one per feature.

Training uses all 10,000 design points, split *by design point* (80/10/10): the sampler queries

the surrogate at parameter values it has never seen, so a split by replicate, which leaves every design point in every split, would measure the wrong thing. The held-out target carries irreducible replicate-to-replicate noise; we estimate it from within-design-point scatter and report accuracy as a multiple of that floor, a figure that is 1.0 for a perfect surrogate.

5.4 Choosing the architecture

We search twelve architectures: plain MLPs of five widths and depths with five activations, and two residual networks, five seeds each (60 fits per model). Selection uses validation loss only; test metrics are reported but never used to choose. Following Lu et al's own warning about cost, we apply a rule fixed in advance: take the best mean validation loss, and choose the cheapest model, by single-query latency, whose validation loss is within one standard error of it. The Gaussian-process baseline is fit to the same data on the nonzero part only, because a GP cannot represent the zero mass, at the original design size of 1,500 points and, for reference, at 300. Table 7 reports the search.

5.5 The inference methods

GPS-ABC. We run the original authors' MATLAB implementation (`ABC_mu2a.m` with their GP code), on our observed data, with the paper's settings: $\xi = 0.3$, $\epsilon = 10^{-8}$, $M = 100$ draws per acceptance decision, ten new design points added at each refinement, 20,000 draws and 5,000 burn-in, an initial design of 1,500 points with five replicates each. One change was needed to run the box of the paper: the released `mut_bMBP_rev.m` draws the number of cells as the sum of `geornd` over a vector with one entry per founder, which in the second stage means about e^{τ} entries, up to 4×10^8 at $\tau = 19.9$, and the MATLAB workers run out of memory. A sum of Z_0 independent geometric variables is negative binomial, so we replace that line by a single `nbinrnd` call; the distribution is identical and the original file is left untouched.

DNN-ABC. The sampler is the same Metropolis–Hastings algorithm with the same priors and proposal family, with the Gaussian process replaced by the trained network. There is no refinement step, since the network is trained once, and no Monte Carlo draws from the surrogate, so the acceptance decision uses the predictive density directly. The paper's step widths $(0.8, 3.5, 3.5, 4)$ are given without their order; applied to the sharper network likelihood, they accept only a few per cent of proposals, against the paper's stated target of 20–40%. We therefore multiply them by a single factor chosen on a pilot run (first six data sets of each case) to bring acceptance into that range.

NPE, and why it is here. An amortized neural posterior estimator learns $p(\theta \mid \bar{d}, s)$ directly from simulator draws (Papamakarios and Murray, 2016; Greenberg et al, 2019), here with `sbi` (Tejero-Cantero et al, 2020). We include it in this study because it tests, with the same simulations, what the other two methods add on top of them. In one dimension there is little to gain by skipping the sampler; in four, the alternative to a surrogate inside an MCMC kernel is to learn the posterior itself, and the question is whether the sampler, and the surrogate's variance model, are needed at all. NPE uses exactly the DNN-ABC training data, the same two features and the same design-point split, so any difference is the method, not the data. It needs no chain, no acceptance step and no ϵ, so it also separates errors that come from the surrogate's variance from errors that come from the kernel. And after training it returns independent posterior draws for a new data set in milliseconds, which none of GPS-ABC, DNN-ABC and exact ABC-MCMC can. It cannot do better than the information in $(\bar{d}, s)$, which makes it a ceiling on what any method using these features can recover. We train on the rows with $\bar{d} > 0$ only. The population minimiser of the NPE objective is $p(\theta \mid x)$ at every x in the support of the training data, whatever its marginal, so discarding the rows where $\log_{10} \bar{d}$ is undefined leaves the posterior at any observed $\bar{d} > 0$ unchanged; every observed data set here has $\bar{d} > 0$. The prior is uniform on the box, to which the paper's $\mathrm{TN}(\cdot, 20)$ priors are flat to within a few per cent. We fit a neural spline flow and a mixture-density network and keep the one with the higher held-out log-probability on unseen design points, never one chosen on observed data.

5.6 Surrogate fit and calibration

Table 7 compares the twelve architectures on held-out design points. Nine of the twelve reach the data's noise floor on both features, with test mean-squared error between 0.91 and 1.02 times the floor estimated from replicate scatter (a ratio just below

Table 7: Architecture search on held-out design points, five seeds each (mean ± s.d.). Accuracy is test MSE as a multiple of the irreducible noise floor (1.0 is perfect); coverage is of the conformal-calibrated 95% interval; AUC is of the zero head. Latency is a single query; the last column is a batch of 10^5. The chosen model is in bold. The GP baseline models $\log_{10} \bar{d}$ on nonzero rows only.

Surrogate	Params	× floor ($\log_{10} \bar{d}$)	× floor ($\log_{10} s$)	Cov. 95%	AUC (zero)	µs/query	ms/10^5
ResNet 3×128 (GELU)	100,357	0.931 ± 0.004	0.940 ± 0.004	0.943	0.9979	2164	4635
MLP 4×64 (GELU)	13,125	0.913 ± 0.010	0.937 ± 0.003	0.944	0.9979	1383	808
MLP 3×128 (SiLU)	34,309	0.914 ± 0.006	0.937 ± 0.005	0.942	0.9979	947	1893
MLP 256–128–64 (GELU)	42,757	0.924 ± 0.017	0.938 ± 0.005	0.944	0.9979	944	1352
ResNet 4×256 (GELU)	528,901	0.950 ± 0.016	0.946 ± 0.008	0.944	0.9978	3851	18919
MLP 128–64 (tanh)	9,221	0.972 ± 0.018	0.980 ± 0.013	0.941	0.9978	858	852
MLP 128–64 (GELU)	9,221	1.020 ± 0.041	0.966 ± 0.011	0.940	0.9978	1062	650
MLP 64–32 (GELU/tanh)	2,565	0.976 ± 0.010	0.979 ± 0.015	0.938	0.9978	969	333
MLP 128–64 (SiLU)	9,221	1.081 ± 0.049	0.993 ± 0.015	0.940	0.9978	890	973
MLP 128–64 (ReLU)	9,221	0.964 ± 0.032	0.959 ± 0.009	0.943	0.9978	891	712
MLP 32–16 (GELU)	773	1.149 ± 0.027	1.041 ± 0.017	0.945	0.9977	1064	168
MLP 16–16 (tanh)	437	1.127 ± 0.061	1.091 ± 0.031	0.941	0.9975	891	149
GP, 300 points ($\log_{10} \bar{d}$ only)	—	2.37	—	0.914	—	15685	—
GP, 1500 points ($\log_{10} \bar{d}$ only)	—	1.45	—	0.915	—	3936	—

one is estimation error in the floor, not a better-than-perfect fit). Only the two smallest networks, with 437 and 773 parameters, are clearly short of it (1.13–1.15 on $\log_{10} \bar{d}$), and beyond about 2,500 parameters capacity buys nothing measurable: an MLP with 2,565 parameters and one with 42,757 are within about five per cent of each other. The residual networks add no accuracy over plain MLPs. Coverage of the conformal-calibrated 95% interval is between 0.938 and 0.945 for every architecture, and the zero head ranks datasets with $\bar{d} = 0$ against $\bar{d} > 0$ almost perfectly (AUC 0.9975–0.9979 for all twelve).

The selection rule picks the 3×128 residual network: it has the best mean validation loss (-3.342, standard error 0.002) and the next best, a 4×64 MLP, is 0.0045 worse, so it alone is within one standard error. We do not read this as a verdict on residual networks. Three plain MLPs of 13,000–43,000 parameters are within 0.02 in validation loss and two to six times faster on large batches, and nothing in what follows depends on the choice. On held-out designs the selected network has test error 0.934 and 0.938 times the floor on $\log_{10} \bar{d}$ and $\log_{10} s$, coverage 0.943 and 0.954 after the split-conformal scale factors (0.989 and 1.034, so the variance heads were already nearly calibrated), and zero-head log-loss 0.054.

The Gaussian-process baseline of GPS-ABC, fit to the original design size of 1,500 points, is 1.45 times the floor on $\log_{10} \bar{d}$ alone (2.37 at 300 points), with 0.915 coverage of its nominal-95% interval (0.914), and it has no way to represent the 30% of datasets with $\bar{d} = 0$. The network is at 0.93 times the floor, and at 0.96 when trained on the same 1,500 design points. This baseline is a diagnostic of the surrogate, not a re-run of GPS-ABC; the latter, with the authors' own code and its refinement step, is reported in Section 5.11.

5.7 Is it the data or the model?

The neural surrogate is trained on 10,000 design points and GPS-ABC starts from 1,500, so a gain over GPS-ABC could come from the surrogate or simply from more simulation. We separate the two by training the chosen network on only the first 1,500 design points of the training split (the original design size), with every other part of the pipeline unchanged. It is 0.964 and 0.982 times the floor on the two features, with coverage 0.946 and zero-head AUC 0.998, against 0.934, 0.938, 0.943 and 0.998 for the full-data network, and its ABC-MCMC posteriors are close to those of the full-data network (Table 8, rows "DNN-ABC (1,500)"; for example the RMSE of $\log_{10} p_2$ is 0.077 against 0.078 in case 1 and 0.086 against 0.088 in case 2). The extra training data buys almost nothing here,

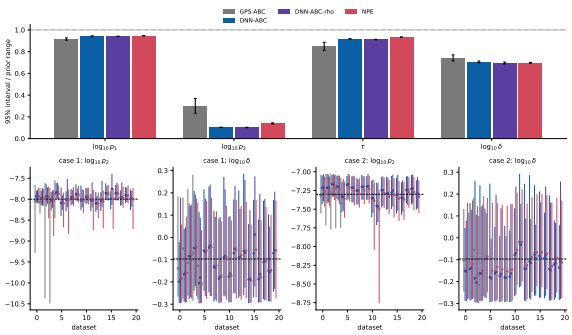


Fig. 3: Study II: what each method learns about each parameter. Top: mean length of the 95% interval as a fraction of the prior range (error bars: standard error over the data sets). Bottom: intervals for $\log_{10} p_2$ and $\log_{10} \delta$ for the first twenty data sets of each case; the dashed line is the truth.

which means that any difference from GPS-ABC at the same design size cannot be attributed to the amount of simulation.

5.8 Parameter recovery

Each method is applied to 100 observed data sets per truth case. Table 8 reports, per parameter, the RMSE of the posterior median against the truth (on the $\log_{10}$ scale for p_1, p_2, δ), the mean length of the 95% credible interval, its coverage of the truth, and the cost per dataset; Figure 3 shows how much of the prior range each method's interval spans.

The data identify two parameters and not the other two. The 95% interval for $\log_{10} p_2$ covers about 10–15% of its prior range and that for $\log_{10} \delta$ about 70%; those for $\log_{10} p_1$ and τ cover 94% and 92%, which is to say the prior. Posterior medians for p_1 and τ sit near the prior's centre, so their RMSE says more about where the truth lies in the box than about the method: the p_1 truth is at the centre of its box, so its RMSE is small (0.08–0.17 in $\log_{10}$ units) for any method that returns the prior, while the τ truth is 4 units from the centre and its RMSE is 2.3–3.8 for all. This is the asymmetry of Lu et al's Figures 3–4, with multimodal p_1 and τ marginals, and it is set by what the pair $(\bar{d}, s)$ carries, not by the method.

5.9 Cost

A DNN-ABC chain of 35,000 iterations (10,000 for adapting the proposal and 5,000 of burn-in, both discarded, then 20,000 retained) takes about 40 seconds on one core; training the surrogate takes about ten minutes. NPE, once trained, returns 20,000 independent draws for a new data set in about four seconds, but training the neural spline flow took 5.6 hours on the shared server (the mixture-density alternative took 49 minutes and had the lower held-out log-probability, −1.734 against −1.678).

5.10 Calibration over prior draws

Coverage at a fixed truth cannot separate a calibrated posterior from an uninformative one: an interval that spans the prior covers any truth, and in Table 8 the intervals for p_1 and τ nearly do. We therefore evaluate calibration by simulation-based calibration (Talts et al, 2018) over truths drawn from the prior. For each of $K = 500$ truths drawn uniformly from the box we simulate a data set of $J = 100$ cultures, compute each method's posterior and record the rank of the truth among 1,000 posterior draws; a calibrated posterior gives uniform ranks and central intervals with their nominal coverage, whatever the identifiability. We also report posterior contraction, the share of prior variance removed, $1 - \overline{\mathrm{Var}}_{\mathrm{post}}/\mathrm{Var}_{\mathrm{prior}}$. NPE is trained on data sets with $\bar{d} > 0$ and so can only be scored on those (364 of the 500); ranks are uniform conditionally on any event defined by the data if the posterior is correct, so the restriction is valid, and we score DNN-ABC on the same 364. GPS-ABC is not included: it would need hundreds of chains, each of which takes days.

Table 9 separates two things that the fixed-truth coverage runs together. NPE is the best calibrated method: its 95% intervals cover 0.942, 0.964, 0.948 and 0.942 of the truths for $\log_{10} p_1$, $\log_{10} p_2$, τ and $\log_{10} \delta$, all within about 1.3 standard errors of nominal, and its ranks are uniform at the 2% level in every cell. The surrogate-in-MCMC methods are sharper and overconfident. DNN-ABC covers 0.885 for $\log_{10} p_2$, nearly six standard errors short of 0.95, and 0.909 and 0.923 for τ and $\log_{10} \delta$. It removes 0.98 of the prior variance of $\log_{10} p_2$ against 0.87 for NPE, which is where the extra sharpness, and the miscalibration, come from; and for τ it removes 0.14 of the prior variance where NPE removes none (−0.01), so that part of what it learns about τ is not in the data. Modelling the correlation of the two features makes the ranks more uniform (χ^2 $p = 0.51$ for $\log_{10} p_2$ and 0.98

for $\log_{10}\delta$, against 0.17 and 0.04), and removes more variance of $\log_{10}\delta$ (0.63 against 0.58, and 0.62 for NPE), but does not restore nominal coverage (0.896, 0.901 and 0.901 for $\log_{10} p_2$, τ and $\log_{10}\delta$). The independence assumption inherited from GPS-ABC is therefore not the main reason for the overconfidence. We have not identified it; candidates are the Gaussian form of the predictive density of the log-features, which NPE does not assume, and the surrogate's own error away from the training points. Rank histograms are in Supplementary Figure S1.

5.11 Comparison with GPS-ABC

We ran the original authors' MATLAB GPS-ABC on the first five of the 100 observed data sets of each case, so that every comparison with it is paired, with the settings of Supplementary Table S4. One chain at the original 20,000 iterations is not feasible at this scale: the Gaussian process is refit, and grows by ten design points, whenever an accept/reject decision is uncertain, so the cost per iteration rises as the chain runs. We therefore analyse every chain at its first 2,000 iterations, discarding the first 500 (the paper's burn-in fraction), and report what that costs. A chain took between 4.4 and 18.6 hours for those 2,000 iterations, 10.6 hours on average (on two cores of a shared 32-core server), so 20,000 iterations would take at least four days; a DNN-ABC chain of 35,000 iterations takes about 40 seconds.

Our GPS-ABC reproduces the published one. Table 4 of Lu et al (2023), averaged over 100 replications, reports 95% intervals for p_2 of $[9.9 \times 10^{-10}, 2.5 \times 10^{-8}]$ in case 1 and $[1.1 \times 10^{-8}, 9.3 \times 10^{-8}]$ in case 2, that is 1.4 and 0.94 in $\log_{10}$ units, and for δ of $[0.52, 1.76]$ and $[0.52, 1.62]$, about 0.5 in $\log_{10}$ units, with posterior medians close to the truth. Our runs of the same sampler give 1.38 and 0.62 for p_2 and 0.49 and 0.41 for δ, with errors of the median of 0.09 and 0.05 for $\log_{10} p_2$. Their interval for τ ($[4.8, 17.8]$) is not comparable to ours because their released code ran it on $[4, 18]$ and not on the paper's $[0.1, 19.9]$ (Section 5.1).

Same accuracy, much shorter intervals. On the identified parameters the posterior medians of the three methods are about equally close to the truth: the RMSE of $\log_{10} p_2$ is 0.085 and 0.054 for GPS-ABC in the two cases, 0.085 and 0.097 for DNN-ABC and 0.068 and 0.066 for NPE, and for $\log_{10}\delta$ it is 0.04 for GPS-ABC against 0.06–0.08 and 0.05–0.07 for the others. With five data sets per case none of these differences is resolved. The intervals differ. GPS-ABC's 95% interval for $\log_{10} p_2$ is 1.38 and 0.62 long, against 0.39 and 0.36 for DNN-ABC (3.6 and 1.7 times longer) and 0.60 and 0.40 for NPE (2.3 and 1.6 times). For δ and for p_1 and τ, whose intervals are close to the prior for every method, the lengths agree. Distances between posteriors show the same thing: the Wasserstein distance from the DNN-ABC posterior for $\log_{10} p_2$ to the GPS-ABC posterior is 0.165 and 0.093 (NPE: 0.140 and 0.073), small beside GPS-ABC's own standard deviation of 0.39 and 0.20, and for δ about 0.05 against 0.13 and 0.11 (Table 11). The neural methods put their posteriors in the same place as GPS-ABC and make them narrower.

How to read the extra width. GPS-ABC's wider interval is a loss of precision, not accuracy. Its surrogate carries the Gaussian-process error (1.45 times the noise floor at the original design size, on $\log_{10}\bar{d}$ alone) into the acceptance step, which inflates the posterior. Whether that inflation is warranted cannot be tested directly, since calibrating GPS-ABC over prior draws would need hundreds of chains. The calibrated method, NPE, lies between the two (0.60 and 0.40), and the simulation-based calibration above shows that DNN-ABC's shorter intervals are a little too short (Table 9); so the honest reading is that GPS-ABC is conservative, DNN-ABC is sharp and mildly overconfident, and NPE is closest to calibrated, with the neural methods costing seconds where GPS-ABC costs days.

6 Discussion

The two studies answer the question of Section 1 with a condition, not a verdict. *A surrogate improves an ABC inference where surrogate error is what limits it, and a neural surrogate removes the cost that limits a GP.* In Study I surrogate error binds: a heteroscedastic network gives tighter intervals than a GP and a flat query cost. In Study II it binds more: the response surface is zero-inflated, spans four decades and has strongly correlated features, and a Gaussian process fit to the original design's 1,500 points sits at 1.45 times the data's noise floor (2.4 at 300 points) while the network sits at 0.93 times it, and at 0.96 when trained on the same 1,500 points. The cost is the other

half: one GPS-ABC chain of 20,000 iterations is projected to take at least four days (linear extrapolation from the 10.6 hours that 2,000 iterations took, with the cost per iteration rising as the chain runs), because the Gaussian process is refit, and grows, whenever an accept/reject decision is uncertain, while a DNN-ABC chain of 35,000 iterations takes about 40 seconds and NPE returns independent draws in four seconds once trained. On the same data sets and against the original authors' own sampler, DNN-ABC and NPE are not distinguishable from GPS-ABC in the accuracy of the posterior median for the identified parameters and give a 95% interval for p_2 that is 1.6 to 3.6 times shorter (Section 5.11); the shorter intervals are sharper than GPS-ABC's and, for DNN-ABC, a little too short. The noise-floor measurement is cheap and tells which regime one is in; we recommend making it before any surrogate comparison.

The control of Section 5.7 qualifies how this should be read. A network trained on only 1,500 design points, the size of the original design, is as good as one trained on 10,000, so the advantage over the GP is not that the network can use more simulation. It is the model class: a hurdle distribution with input-dependent variances for a pair of features.

The second finding concerns identifiability. The summary statistic identifies p_2, and δ loosely, and says almost nothing about p_1 or τ: the 95% intervals for those two span 94% and 92% of the prior range. An NPE trained directly on simulator draws, with no surrogate and no acceptance kernel, and a surrogate substituted into an MCMC kernel agree on this, which is evidence that the ceiling belongs to the summary statistic and not to either method. A practical consequence is that a better emulator cannot repair a weakly identified parameter; a richer summary than $(\bar{d}, s)$ is the lever.

The third finding is about calibration. Split-conformal calibration is a strong, distribution-free guarantee and it holds for the surrogate's regression problem (coverage 0.943 and 0.954 for the two features). It says nothing about the coverage of the posterior built on it. Over truths drawn from the prior, NPE is calibrated (95% intervals cover 0.94–0.96) but less sharp on p_2 (it removes 0.87 of the prior variance, against 0.98 for DNN-ABC), while the surrogate-based samplers are sharper and overconfident (0.885 for p_2, nearly six standard errors below nominal, and 0.90–0.93 for the others). At a fixed truth, the two look alike. We therefore recommend simulation-based calibration over prior draws, together with posterior contraction, as the routine check for any surrogate-based likelihood-free sampler. The reason for the overconfidence is open. We found that the two features are correlated (0.61) and that GPS-ABC's treatment of them as independent is wrong in that sense; modelling the correlation improves the held-out joint likelihood and the uniformity of the ranks but not the coverage (0.896 for p_2), so it is not the main cause. The Gaussian form of the predictive density, which NPE does not assume, and surrogate error away from the training points are the remaining candidates.

Two architectural lessons recur. In Study I the dominant factor was not the activation function but the presence of BatchNorm (Ioffe and Szegedy, 2015): its per-minibatch statistics are counterproductive for a smooth regression on a bounded, one-sided domain, and removing it improved the fit eleven-fold. In Study II no architectural choice mattered much: nine of twelve candidates sit within 10% of the noise floor, residual networks add nothing over plain MLPs, and only the two smallest networks ($\leq$ 773 parameters) are clearly worse. The practical lesson is to let the symmetry of the input choose the architecture family (Radev et al, 2020) and to measure the floor before searching.

Limitations. This is a simulation study under known ground truth, with no application to real fluctuation-experiment data as in Lu et al (2023); only that can speak to misspecification. Study II has two truth cases with 100 data sets each for the neural methods. GPS-ABC is run on five of those data sets per case and for 2,000 (the first 500 discarded) iterations per chain, not the original study's 20,000, because one chain takes days; its comparison with the neural methods is therefore indicative, and what it establishes firmly is its cost. The fast simulator is that of the original paper; its p is not the per-division probability of the exact simulator (Section 5.1), which does not affect any comparison here but matters for interpreting the estimates. Our data generator is tested against the original authors' released training data at their design points, not against their observed data sets, which are not public. Study I uses 40 replicates against Lu et al's 100, and we report Monte Carlo standard errors throughout so that

unresolved differences are not over-interpreted. The prior is bounded to the original study's box. NPE is trained on data sets with $\bar{d} > 0$ and so does not answer for data with no mutants at all. Finally, why the surrogate-based samplers are overconfident, and whether a kernel-side correction restores nominal coverage, is left open.

7 Reproducibility

Code and data. All code, data, trained weights and the exact configuration for every table and figure are in the public repository https://github.com/sletch1/DNN-ABC-mutation-rates. The one-dimensional and two-stage training sets are committed as plain-text CSV files with self-documenting headers; the two-stage set is regenerated in about a minute by `4D/datagen/gen_training.py`, and its observed data sets by `4D/datagen/gen_observed.py`. Every random stream is seeded, and the output of the data generator does not depend on the number of cores. The original authors' released MATLAB code is included unmodified; the one function we replace for the GPS-ABC runs, and the checkpointing copy of their sampler, are in `4D/gps_abc/patched/` with the reasons stated at the top of each file, and the places where the published paper and the released code disagree, with the side we took, are listed in `paper_code_discrepancy.md`.

Environment. All code is Python (`numpy`, `pandas`, `scipy`, `matplotlib`, `scikit-learn` for the GP baseline, `torch` for the surrogates, `sbi` for NPE; minimum versions in `requirements.txt`) except the GPS-ABC baseline, which is the authors' MATLAB (R2022b with the Statistics and Parallel Computing toolboxes). No GPU is required. The heavy steps were run on a 32-core server; on it the architecture search takes under an hour, the DNN-ABC chains for 200 data sets about thirty minutes, NPE training about six hours and each GPS-ABC chain days.

Running it. Study I has a single entry point, `1D/run_all.py`. Study II is a short sequence of scripts documented in `4D/README.md`: data generation, the architecture search (`server/run_search_4d.sh`), DNN-ABC, NPE and calibration (`server/run_study2.sh`), the matched-budget and correlated-feature variants (`server/run_matched.sh`, `server/run_corr.sh`), GPS-ABC (`server/run_gps_case.sh`) and the analysis (`analysis/study2_analysis.py`, which writes every table and figure that the manuscript includes). All Study II tables are generated as LaTeX fragments by `analysis/make_tables_tex.py`, so no number in them is copied by hand.

Validation scripts. `4D/tests/validate_vs_published.py` tests the data generator against the original authors' released training data (it needs those files, which are not redistributed); `4D/tests/test_sim.py` checks the simulators against each other and the design for completeness.

8 Conclusion

A heteroscedastic neural surrogate, calibrated by split-conformal prediction and substituted for the Gaussian process inside Lu et al's ABC-MCMC sampler, gives tighter intervals at matched accuracy and a 68 to 2,381-fold speedup over the exact sampler in the one-dimensional benchmark. In the four-parameter two-stage model, which is the main study and is run in the original study's own regime, the surrogate has to represent a response that is exactly zero for 30% of data sets, spans four decades and has two strongly correlated features; a zero-aware network does this at the data's noise floor where a Gaussian process fit to the original design's 1,500 points is 1.45 times above it, and it needs no more than those 1,500 design points to do so. Against the original authors' own sampler, run on the same data sets, it matches GPS-ABC's accuracy on the identified parameters with 95% intervals for p_2 that are 1.7 to 3.6 times shorter, in about forty seconds per data set where a GPS-ABC chain takes days; the shorter intervals are mildly overconfident. An amortized NPE trained on the same simulations agrees with it on what the data identify: p_2 well, δ loosely, and p_1 and τ not at all from the pair $(\bar{d}, s)$. NPE is the better calibrated of the neural approaches over truths drawn from the prior, and the surrogate-based samplers are sharper and mildly overconfident.

The general lessons are three. Measure the noise floor before comparing architectures or surrogates, because it predicts whether a comparison can report anything. Do not read a surrogate's regression calibration as the calibration of the posterior built on it; check the latter with simulation-based

calibration over prior draws and posterior contraction, because fixed-truth coverage saturates at one for weakly identified parameters. And do not expect a better emulator to repair a parameter the data do not identify. These conclusions should apply beyond this model to ABC procedures built on a regression surrogate for an expensive simulator.

Declarations

Data availability. The ground-truth data sets analysed in this study (Section 4 for the one-dimensional constant-rate model and Section 5.1 for the four-parameter two-stage model) are committed as plain-text CSV files in the code repository accompanying this paper, https://github.com/sletch1/DNN-ABC-mutation-rates. [PLACEHOLDER: insert the specific release tag or archived-version DOI corresponding to this submission once one has been minted.]

Code availability. All code used to produce the results in this paper (surrogate training, ABC-MCMC sampling, Monte Carlo standard error computation, and figure/table generation) is available in the same repository as above, https://github.com/sletch1/DNN-ABC-mutation-rates. Study I has a single entry point, `run_all.py`; Study II is a short sequence of scripts (see Section 7). [PLACEHOLDER: insert the release tag or archived-version DOI corresponding to this submission once one has been minted.]

Competing interests. [PLACEHOLDER: the authors will state any competing interests here prior to submission.]

Funding. [PLACEHOLDER: the authors will state funding sources, if any, here prior to submission.]

Author contributions. [DRAFT, please confirm or edit before submission: S.L. conceived and implemented the surrogate methods, ran all simulation studies and analyses, and wrote the manuscript. H.Z. and X.W. supervised the research and critically reviewed the manuscript. All authors read and approved the final manuscript.]

References

Beaumont MA, Zhang W, Balding DJ (2002) Approximate Bayesian computation in population genetics. Genetics 162(4):2025–2035. https://doi.org/10.1093/genetics/162.4.2025

Cranmer K, Brehmer J, Louppe G (2020) The frontier of simulation-based inference. Proceedings of the National Academy of Sciences 117(48):30055–30062. https://doi.org/10.1073/pnas.1912789117

Elfwing S, Uchibe E, Doya K (2018) Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. Neural Networks 107:3–11. https://doi.org/10.1016/j.neunet.2017.12.012

Fan Y, White S (2026) Neural posterior estimation on exponential random graph models: evaluating bias and implementation challenges. Statistics and Computing 36. https://doi.org/10.1007/s11222-026-10896-8

Greenberg DS, Nonnenmacher M, Macke JH (2019) Automatic posterior transformation for likelihood-free inference. In: Proceedings of the 36th International Conference on Machine Learning (ICML), pp 2404–2414

Gutmann MU, Corander J (2016) Bayesian optimization for likelihood-free inference of simulator-based statistical models. Journal of Machine Learning Research 17(125):1–47

Hendrycks D, Gimpel K (2016) Gaussian error linear units (GELUs). arXiv preprint arXiv:160608415

Hermans J, Delaunoy A, Rozet F, et al (2022) A crisis in simulation-based inference? beware, your posterior approximations can be unfaithful. Transactions on Machine Learning Research

Ioffe S, Szegedy C (2015) Batch normalization: Accelerating deep network training by reducing internal covariate shift. In: Proceedings of the 32nd International Conference on Machine Learning (ICML), pp 448–456

Järvenpää M, Corander J (2023) On predictive inference for intractable models via approximate Bayesian computation. Statistics and Computing 33(2):42. https://doi.org/10.1007/s11222-022-10163-6

Järvenpää M, Gutmann MU, Vehtari A, et al (2021) Parallel Gaussian process surrogate Bayesian inference with noisy likelihood evaluations. Bayesian Analysis 16(1):147–178. https://doi.org/10.1214/20-BA1200

Lei J, G'Sell M, Rinaldo A, et al (2018) Distribution-free predictive inference for regression. Journal of the American Statistical Association 113(523):1094–1111. https://doi.org/10.1080/01621459.2017.1307116

Liao X, Yan X, Xia W, et al (2010) A fast optimal Latin hypercube design for Gaussian process regression modeling. In: 3rd International Workshop on Advanced Computational Intelligence (IWACI), https://doi.org/10.1109/IWACI.2010.5585160

Lu R, Zhu H, Wu X (2023) Estimating mutation rates in a Markov branching process using approximate Bayesian computation. Journal of Theoretical Biology 565:111467. https://doi.org/10.1016/j.jtbi.2023.111467

Marin JM, Pudlo P, Robert CP, et al (2012) Approximate Bayesian computational methods. Statistics and Computing 22(6):1167–1180. https://doi.org/10.1007/s11222-011-9288-2

Marjoram P, Molitor J, Plagnol V, et al (2003) Markov chain Monte Carlo without likelihoods. Proceedings of the National Academy of Sciences 100(26):15324–15328. https://doi.org/10.1073/pnas.0306899100

Meeds E, Welling M (2014) GPS-ABC: Gaussian process surrogate approximate Bayesian computation. In: Proceedings of the Thirtieth Conference on Uncertainty in Artificial Intelligence (UAI), pp 593–602

Morris TP, White IR, Crowther MJ (2019) Using simulation studies to evaluate statistical methods. Statistics in Medicine 38(11):2074–2102. https://doi.org/10.1002/sim.8086

Papamakarios G, Murray I (2016) Fast ϵ-free inference of simulation models with Bayesian conditional density estimation. In: Advances in Neural Information Processing Systems 29 (NeurIPS), pp 1028–1036

Radev ST, Mertens UK, Voss A, et al (2020) BayesFlow: Learning complex stochastic models with invertible neural networks. arXiv preprint arXiv:200306281

Talts S, Betancourt M, Simpson D, et al (2018) Validating Bayesian inference algorithms with simulation-based calibration. arXiv preprint arXiv:180406788

Tejero-Cantero A, Boelts J, Deistler M, et al (2020) sbi: A toolkit for simulation-based inference. Journal of Open Source Software 5(52):2505. https://doi.org/10.21105/joss.02505

Wilkinson RD (2014) Accelerating ABC methods using Gaussian processes. In: Proceedings of the Seventeenth International Conference on Artificial Intelligence and Statistics (AISTATS), pp 1015–1023

Wood SN (2010) Statistical inference for noisy nonlinear ecological dynamic systems. Nature 466:1102–1104. https://doi.org/10.1038/nature09319

Table 8: Study II: recovery of $(\log_{10} p_1, \log_{10} p_2, \tau, \log_{10} \delta)$ from 100 observed data sets per case. RMSE is of the posterior median; interval length is the mean 95% interval length (prior ranges: 4, 4, 19.8 and 0.60); the last column is the cost per dataset, for NPE the sampling time only (training took 5.6 h once; DNN-ABC training takes minutes). Truth: $p_1 = 10^{-9}$, $\tau = 14$, $\delta = 0.8$; case 1 $p_2 = 10^{-8}$, case 2 $p_2 = 5 \times 10^{-8}$. Case labels are in the first column.

Case	Method	Parameter	RMSE	95% CI length	Coverage	s/dataset
1	GPS-ABC ($n = 5$)	$\log_{10} p_1$	0.363	3.675	1.00	48,545.5
		$\log_{10} p_2$	0.085	1.383	1.00	
		τ	2.402	17.278	1.00	
		$\log_{10} \delta$	0.041	0.487	1.00	
	DNN-ABC ($n = 100$)	$\log_{10} p_1$	0.119	3.777	1.00	42.3
		$\log_{10} p_2$	0.078	0.431	0.95	
		τ	3.836	18.385	1.00	
		$\log_{10} \delta$	0.061	0.416	1.00	
	DNN-ABC (1,500) ($n = 100$)	$\log_{10} p_1$	0.173	3.768	1.00	47.1
		$\log_{10} p_2$	0.077	0.446	0.97	
		τ	3.527	18.616	1.00	
		$\log_{10} \delta$	0.057	0.437	1.00	
	DNN-ABC-ρ ($n = 100$)	$\log_{10} p_1$	0.173	3.762	1.00	39.7
		$\log_{10} p_2$	0.081	0.429	0.97	
		τ	3.554	18.362	1.00	
		$\log_{10} \delta$	0.061	0.423	1.00	
	DNN-ABC-ρ (1,500) ($n = 100$)	$\log_{10} p_1$	0.205	3.764	1.00	39.7
		$\log_{10} p_2$	0.085	0.455	0.96	
		τ	3.546	18.562	1.00	
		$\log_{10} \delta$	0.060	0.417	1.00	
	NPE ($n = 100$)	$\log_{10} p_1$	0.080	3.769	1.00	4.1
		$\log_{10} p_2$	0.071	0.608	1.00	
		τ	3.491	18.666	1.00	
		$\log_{10} \delta$	0.060	0.401	1.00	
2	GPS-ABC ($n = 5$)	$\log_{10} p_1$	0.242	3.656	1.00	27,939.3
		$\log_{10} p_2$	0.054	0.622	1.00	
		τ	3.633	17.725	1.00	
		$\log_{10} \delta$	0.038	0.413	1.00	
	DNN-ABC ($n = 100$)	$\log_{10} p_1$	0.127	3.773	1.00	38.6
		$\log_{10} p_2$	0.088	0.401	0.95	
		τ	3.282	17.996	1.00	
		$\log_{10} \delta$	0.056	0.434	1.00	
	DNN-ABC (1,500) ($n = 100$)	$\log_{10} p_1$	0.112	3.777	1.00	47.0
		$\log_{10} p_2$	0.086	0.432	0.99	
		τ	3.324	18.069	1.00	
		$\log_{10} \delta$	0.052	0.451	1.00	
	DNN-ABC-ρ ($n = 100$)	$\log_{10} p_1$	0.152	3.768	1.00	40.3
		$\log_{10} p_2$	0.089	0.393	0.98	
		τ	3.436	17.752	1.00	
		$\log_{10} \delta$	0.055	0.415	1.00	
	DNN-ABC-ρ (1,500) ($n = 100$)	$\log_{10} p_1$	0.189	3.776	1.00	38.5
		$\log_{10} p_2$	0.088	0.419	0.99	
		τ	3.364	17.887	1.00	
		$\log_{10} \delta$	0.054	0.442	1.00	
	NPE ($n = 100$)	$\log_{10} p_1$	0.124	3.801	1.00	4.5
		$\log_{10} p_2$	0.077	0.508	0.99	
		τ	2.299	18.321	1.00	
		$\log_{10} \delta$	0.042	0.439	1.00	

Table 9: Simulation-based calibration over 500 prior draws (364 with $\bar{d} > 0$; binomial s.e. of a coverage near 0.95 is 0.011). Coverage is of central 50, 90 and 95% intervals; the rank p-value is of a χ^2 test of uniformity on ten bins; contraction is the share of prior variance removed.

Method	Parameter	Cov. 50%	Cov. 90%	Cov. 95%	Rank χ^2 p	Contraction
DNN-ABC	$\log_{10} p_1$	0.481	0.874	0.934	0.363	0.061
	$\log_{10} p_2$	0.486	0.846	0.885	0.174	0.979
	τ	0.431	0.860	0.909	0.430	0.138
	$\log_{10} \delta$	0.563	0.882	0.923	0.037	0.583
DNN-ABC-ρ	$\log_{10} p_1$	0.470	0.857	0.926	0.103	0.076
	$\log_{10} p_2$	0.511	0.852	0.896	0.514	0.976
	τ	0.464	0.843	0.901	0.354	0.146
	$\log_{10} \delta$	0.525	0.871	0.901	0.980	0.633
NPE	$\log_{10} p_1$	0.486	0.885	0.942	0.838	0.049
	$\log_{10} p_2$	0.505	0.909	0.964	0.019	0.869
	τ	0.503	0.901	0.948	0.593	-0.007
	$\log_{10} \delta$	0.538	0.871	0.942	0.055	0.620

Table 10: Study II, paired comparison on the first five data sets of each case. GPS-ABC chains are analysed at 2,000 iterations with the first 500 discarded; the last column is its wall-clock time for those 2,000 iterations, and for the others the time per data set (NPE: sampling only). RMSE is of the posterior median, interval length is the mean 95% interval length on the $\log_{10}$ scale (τ in time units). Coverage of a single truth over five data sets takes values in steps of 0.2.

Case	Method	Parameter	RMSE	95% CI length	Coverage	s/dataset
1	GPS-ABC ($n = 5$)	$\log_{10} p_1$	0.363	3.675	1.00	48,545.5
		$\log_{10} p_2$	0.085	1.383	1.00	
		τ	2.402	17.278	1.00	
		$\log_{10} \delta$	0.041	0.487	1.00	
	DNN-ABC ($n = 5$)	$\log_{10} p_1$	0.117	3.772	1.00	41.2
		$\log_{10} p_2$	0.085	0.389	0.80	
		τ	3.555	18.243	1.00	
		$\log_{10} \delta$	0.077	0.378	1.00	
	DNN-ABC (1,500) ($n = 5$)	$\log_{10} p_1$	0.169	3.769	1.00	47.9
		$\log_{10} p_2$	0.078	0.390	1.00	
		τ	3.536	18.516	1.00	
		$\log_{10} \delta$	0.071	0.411	1.00	
	DNN-ABC-ρ ($n = 5$)	$\log_{10} p_1$	0.230	3.739	1.00	40.5
		$\log_{10} p_2$	0.088	0.380	1.00	
		τ	3.746	18.302	1.00	
		$\log_{10} \delta$	0.066	0.403	1.00	
	DNN-ABC-ρ (1,500) ($n = 5$)	$\log_{10} p_1$	0.210	3.760	1.00	38.8
		$\log_{10} p_2$	0.088	0.434	0.80	
		τ	3.336	18.560	1.00	
		$\log_{10} \delta$	0.082	0.403	1.00	
	NPE ($n = 5$)	$\log_{10} p_1$	0.095	3.768	1.00	4.1
		$\log_{10} p_2$	0.068	0.602	1.00	
		τ	3.424	18.638	1.00	
		$\log_{10} \delta$	0.073	0.384	1.00	
2	GPS-ABC ($n = 5$)	$\log_{10} p_1$	0.242	3.656	1.00	27,939.3
		$\log_{10} p_2$	0.054	0.622	1.00	
		τ	3.633	17.725	1.00	
		$\log_{10} \delta$	0.038	0.413	1.00	
	DNN-ABC ($n = 5$)	$\log_{10} p_1$	0.134	3.776	1.00	35.3
		$\log_{10} p_2$	0.097	0.355	1.00	
		τ	3.165	18.215	1.00	
		$\log_{10} \delta$	0.058	0.454	1.00	
	DNN-ABC (1,500) ($n = 5$)	$\log_{10} p_1$	0.129	3.752	1.00	43.1
		$\log_{10} p_2$	0.085	0.395	1.00	
		τ	2.676	18.046	1.00	
		$\log_{10} \delta$	0.058	0.478	1.00	
	DNN-ABC-ρ ($n = 5$)	$\log_{10} p_1$	0.131	3.766	1.00	40.3
		$\log_{10} p_2$	0.094	0.358	1.00	
		τ	3.344	17.778	1.00	
		$\log_{10} \delta$	0.064	0.400	1.00	
	DNN-ABC-ρ (1,500) ($n = 5$)	$\log_{10} p_1$	0.082	3.781	1.00	40.4
		$\log_{10} p_2$	0.095	0.376	1.00	
		τ	4.140	17.323	1.00	
		$\log_{10} \delta$	0.071	0.335	1.00	
	NPE ($n = 5$)	$\log_{10} p_1$	0.081	3.794	1.00	4.5
		$\log_{10} p_2$	0.066	0.397	1.00	
		τ	1.988	18.191	1.00	
		$\log_{10} \delta$	0.045	0.440	1.00	

Table 11: Wasserstein distance of each posterior from the GPS-ABC posterior for the same data set, per parameter ($\log_{10}$ units for p_1, p_2, δ; mean $\pm$ s.e. over five data sets).

Case	Method	Parameter	W_1 to GPS-ABC	n
1	DNN-ABC	$\log_{10}\delta$	0.055 ± 0.013	5
		$\log_{10} p_1$	0.225 ± 0.055	5
		$\log_{10} p_2$	0.165 ± 0.021	5
		τ	1.374 ± 0.231	5
	DNN-ABC-ρ	$\log_{10}\delta$	0.049 ± 0.010	5
		$\log_{10} p_1$	0.230 ± 0.061	5
		$\log_{10} p_2$	0.165 ± 0.025	5
		τ	1.414 ± 0.247	5
	NPE	$\log_{10}\delta$	0.059 ± 0.011	5
		$\log_{10} p_1$	0.203 ± 0.045	5
		$\log_{10} p_2$	0.140 ± 0.020	5
		τ	1.313 ± 0.366	5
2	DNN-ABC	$\log_{10}\delta$	0.048 ± 0.012	5
		$\log_{10} p_1$	0.191 ± 0.041	5
		$\log_{10} p_2$	0.093 ± 0.016	5
		τ	1.170 ± 0.202	5
	DNN-ABC-ρ	$\log_{10}\delta$	0.051 ± 0.017	5
		$\log_{10} p_1$	0.189 ± 0.037	5
		$\log_{10} p_2$	0.094 ± 0.017	5
		τ	1.310 ± 0.414	5
	NPE	$\log_{10}\delta$	0.040 ± 0.009	5
		$\log_{10} p_1$	0.201 ± 0.048	5
		$\log_{10} p_2$	0.073 ± 0.012	5
		τ	1.351 ± 0.255	5